Exemple subiecte examen

Programare Orientată pe Obiecte

Exemplu lant apeluri

Programare Orientată pe Obiecte

Apelul în lanţ (implicit) al constructorilor

```
class Demo{
   int  value1;
   int  value2;

   Demo(){
      value1 = 1;
      value2 = 2;
      System.out.println("Inside 1st Parent Constructor");
   }
   Demo ( int a ) {
      value1 = a;
      System.out.println("Inside 2nd Parent Constructor");
   }
  public void display(){
     System.out.println("Value1 === "+value1);
     System.out.println("Value2 === "+value2);
  }
}
```

Lanţul de apeluri al constructorilor

```
class DemoChild extends Demo{
    int value3;
    int value4;
    DemoChild(){
          super(5);
           value3 = 3;
           value4 = 4;
          System.out.println("Inside the 1st Constructor of Child");
    }
    DemoChild (int a){
           this();
           value3 = a;
           System.out.println("Inside the 2nd Constructor of Child");
    }

    public void display() {…}

    public static void main(String args[]){
            DemoChild d2 = new DemoChild (11);
            d2.display();
      }
}
```

Lanţul de apeluri al constructorilor

Inside 1st Parent Constructor
Inside the 2nd Constructor of Child
Value1 === 1
Value2 === 2
Value3 === 11
Value4 === 0

// decomentare *this*
Inside 1st Parent Constructor
Inside the 1st Constructor of Child
Inside the 2nd Constructor of Child
Value1 === 1
Value2 === 2
Value3 === 11
Value4 === 4

// decomentare şi *super ???*

Lanţul de apeluri al constructorilor

Inside 1st Parent Constructor
Inside the 2nd Constructor of Child
Value1 === 1
Value2 === 2
Value3 === 11
Value4 === 0

// decomentare *this*
Inside 1st Parent Constructor
Inside the 1st Constructor of Child
Inside the 2nd Constructor of Child
Value1 === 1
Value2 === 2
Value3 === 11
Value4 === 4

// decomentare şi *super*
Inside 2nd Parent Constructor
Inside the 1st Constructor of Child
Inside the 2nd Constructor of Child
Value1 === 5
Value2 === 0
Value3 === 11
Value4 === 4

Legare statică/dinamică – static/dynamic binding în Java

Programare Orientată pe Obiecte

Ce va afişa următorul program?

```
class TestEgal{
    public boolean equals ( TestEgal other ) {
        System.out.println( "In equals din TestEgal" );   return false;
    }
// public boolean equals ( Object other ) – mostenita de la Object
public static void main( String [] args ) {
        Object t1 = new TestEgal(), t2 = new TestEgal();
        TestEgal t3 = new TestEgal();
        Object o1 = new Object();
        int count = 0;
        System.out.println( count++ ); // afiseaza 0
        t1.equals( t2 ) ;
        System.out.println( count++ ); // afiseaza 1
        t1.equals( t3 );
        System.out.println( count++ ); // afiseaza 2
        t3.equals( o1 );
        System.out.println( count++ ); // afiseaza 3
        t3.equals(t3);
        System.out.println( count++ ); // afiseaza 4
        t3.equals(t2);
    }
}
```

Rezolvare

Output:

0

1

2

3

In equals din TestEgal

4

Atenţie! Este vorba de overload nu override!

Metoda *public boolean equals (TestEgal other)*

Nu supradefineşte metoda *public boolean equals (Object other)* din clasa Object, ci o supraîncarcă!

Deci, static binding! Singurul loc în care se apleează equals din TestEgal este **t3.equals(t3);** !

Exerciţiu propus

```
class Ana {
        public void print(Ana p) {
           System.out.println("Ana 1\n");
        }
}
class Mihai extends Ana {
        public void print(Ana p) {
           System.out.println("Mihai 1\n");
        }
        public void print(Mihai l) {
           System.out.println("Mihai 2\n");
        }
}
class Dana extends Mihai {
        public void print(Ana p) {
           System.out.println("Dana 1\n");
        }
        public void print(Mihai l) {
           System.out.println("Dana 2\n");
        }
        public void print(Dana b) {
           System.out.println("Dana 3\n");
        }
}
```

Exerciţiu propus

```
public class Test{
    public static void main (String [] args) {
       Mihai stud1 = new Dana();
        Ana stud2 = new Mihai();
        Ana stud3 = new Dana();
        Dana stud4 = new Dana();
        Mihai stud5 = new Mihai();
 1  stud1.print(new Ana()); // Dana 1
 2  ((Dana)stud1).print(new Mihai()); // Dana2
 3  ((Mihai)stud2).print(new Ana()); // Mihai 1
 4  stud2.print(new Dana()); // Mihai 1
 5  stud2.print(new Mihai()); // Mihai 1
 6  stud3.print(new Dana()); // Dana 1
 7  stud3.print(new Ana()); // Dana 1
 8  stud3.print(new Mihai()); // Dana 1
 9  ((Dana)stud3).print(new Mihai()); // Dana 2
10  ((Dana)stud3).print(new Dana()); // Dana 3
11  stud4.print(new Dana()); // Dana 3
12  stud4.print(new Ana()); // Dana 1
13  stud4.print(new Mihai()); // Dana 2
14  stud5.print(new Dana()); // Mihai 2
15  stud5.print(new Mihai()); // Mihai 2
16  stud5.print(new Ana());  // Mihai 1
17  ((Ana)stud5).print(new Mihai()); // Mihai 1
} }
```

Ierarhie

Ana – print (Ana)

|

Mihai – print (Ana), print (Mihai)

|

Dana – print (Ana), print (Mihai), print (Dana)

Tip – nume -> obiect

- **Mihai - stud1 -> Dana**
- **Ana - stud2 -> Mihai**
- **Ana - stud3 -> Dana**
- **Dana - stud4 -> Dana**
- **Mihai - stud5 -> Mihai**

Output

1 Dana 1
2 Dana 2
3 Mihai 1
4 **Mihai 1**
5 **Mihai 1**
6 **Dana 1**
7 **Dana 1**
8 **Dana 1**
9 *Dana 2*
10 *Dana 3*
11 *Dana 3*
12 *Dana 1*
13 *Dana 2*
14 **Mihai 2**
15 Mihai 2
16 Mihai 1
17 Mihai 1

Explicaţii

1. stud1.print(new Ana())

   stud1 -> *Dana* apelează *Dana.print(Ana*)
2. ((Dana)stud1).print(new Mihai());

   stud1 -> *Dana* apelează *Dana.print(new* Mihai())
3. ((Mihai)stud2).print(new Ana());

   stud2 -> *Mihai* apelează *Mihai.print(new* Ana());
4. stud2.print(new Dana());

   stud2 este declarat *Ana*. Atunci când compilatorul se uită să vadă ce poate apela găseşte metoda *print(Ana*) din clasa *Ana*. La execuţie, stud2 este un *Mihai* aşa că va apela metoda p*rint(Ana*) din clasa *Mihai*.
5. Samd